Low Level Design on

Marketing Planning Assistant Agent

Course Name – Agentic AI

Institution Name – Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

S.No.	Student Name	Enrolment No.
1	SANIYA KHAN	EN22IT301091
2	VEDANT MEENA	EN22IT301120
3	GAURAV RAUNDHALE	EN22IT301036
4	SOHAM SINGH KUSHWAHA	EN22IT301104

Group Name – Group 08D12

Project Number- AAI-46

Industry Mentor Name – *Mr.Vaibhav*

University Mentor Name – Prof. Ajeet Singh Rajput

Academic Year – 2022-26

Contents

1. Introduction 3

1.1 Scope of the Document 3

1.2 Intended Audience.....3

1.3 System Overview 4

2. System Design 5

2.1 Application Architecture 5

2.2 Process Flow - Step-by-Step Execution Flow 9

2.3 Information Flow 9

2.4 Component Design 10

2.5 Key Design Considerations.....12

2.6 API Catalogue.....12

3. References 12

1. Introduction

The Marketing Planning Assistant Agent is an advanced Agentic AI system designed to automate and optimize marketing planning tasks. It leverages Large Language Models (LLMs) along with frameworks like LangChain or CrewAI to simulate intelligent decision-making.

The system accepts high-level marketing goals and transforms them into structured, executable plans through multi-step reasoning. It demonstrates how AI agents can independently decompose problems, validate resources, and generate efficient execution schedules.

1.1 Scope of the Document

This document provides a detailed overview of the system design, architecture, workflow, and components of the Marketing Planning Assistant Agent.

It covers:

- System architecture and design
- Execution and information flow
- Component-level breakdown
- API structure and interactions

1.2 Intended Audience

This document is intended for:

- Students and academic evaluators
- Developers and AI enthusiasts
- Researchers in Agentic AI and automation systems

It assumes basic knowledge of Python, APIs, and AI/ML concepts.

1.3 System Overview

The system acts as an intelligent planner that:

- Accepts a marketing goal (e.g., competitor analysis)
- Breaks it into smaller actionable tasks
- Validates tool/resource availability
- Manages dependencies between tasks
- Generates an optimized execution plan

The system showcases autonomous decision-making using a reasoning loop, making it a practical implementation of Agentic AI.

2. System Design

2.1 Application Architecture

---

1. Presentation Layer (UI)

- Built using **Streamlit / Flask / CLI interface**
- Handles user interaction and input collection
- Accepts high-level marketing goals (e.g., *"Competitor Ads Analysis"*)
- Displays generated plans, task lists, and schedules in a readable format

Key Functions:

- Input validation
- User-friendly visualization of output
- Interaction with backend APIs

---

2. Application Layer (Agent Orchestration Layer)

- Implements the **core business logic of the system**
- Manages the **Planner Agent workflow and reasoning loop**
- Coordinates communication between all modules

Responsibilities:

- Goal interpretation
- Task orchestration

- Managing multi-step reasoning (ReAct / Chain-of-Thought)
- Handling feedback from tool validation

---

3. Planning Layer (Core Agent Modules)

- Contains the **main intelligence components** of the system
- Responsible for transforming goals into structured plans

Includes:

- **Planner Agent** → Controls decision-making
- **Task Decomposition Module** → Breaks goals into tasks
- **Dependency Manager** → Orders tasks logically
- **Execution Planner** → Generates final schedule

Key Features:

- Autonomous planning
- Dependency-aware sequencing
- Iterative reasoning

---

4. Tool Layer (Resource & Validation Layer)

- Simulates or integrates external tools and APIs
- Ensures required resources are available before execution

Responsibilities:

- Tool/API availability checking

- Mock tool simulation (e.g., Ads API, Analytics API)
- Returning feedback to the agent

Purpose:

Prevents invalid plans and improves realism of execution

---

5. AI Layer (LLM Integration Layer)

- Powered by **LLM API (OpenAI / Gemini / others)**
- Acts as the **brain of the Planner Agent**

Responsibilities:

- Natural language understanding
- Task reasoning and generation
- Decision-making support
- Plan optimization

---

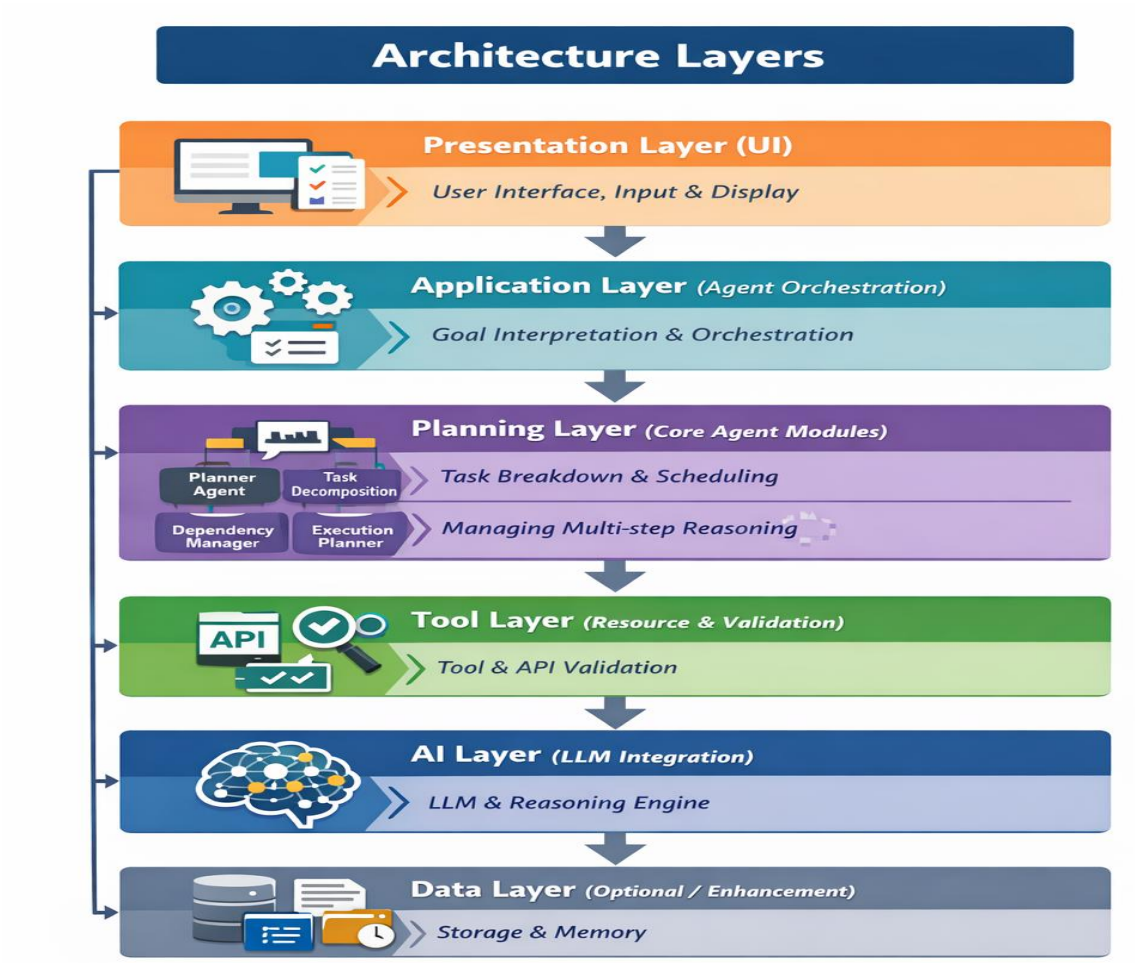
6. Data Layer (Optional / Enhancement Layer)

- Stores intermediate data, memory, or past plans
- Can use:
  - Vector databases (FAISS)
  - JSON storage
  - Logs/history

Functions:

- Context retention (memory)
- Reusability of previous plans
- Performance improvement

Architecture Flow Diagram



2.2 Process Flow - Step-by-Step Execution Flow

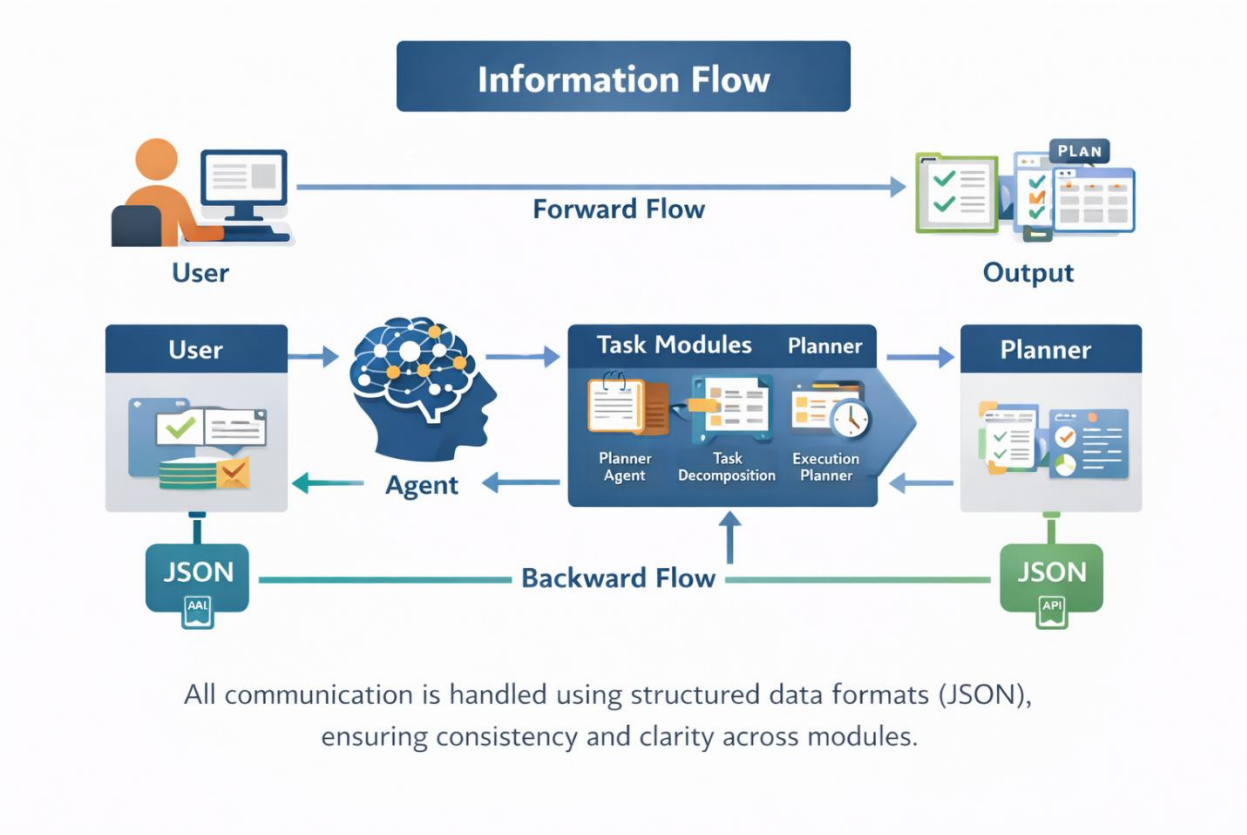
1. User provides a high-level marketing goal
2. Planner Agent initializes reasoning loop
3. Goal is decomposed into smaller tasks
4. Required tools and resources are identified
5. Tool Validator checks availability (mock APIs)
6. Dependencies between tasks are identified
7. Tasks are ordered logically
8. Final execution plan and schedule are generated

2.3 Information Flow

Data Flow Description

1. The information flow in the system is bi-directional:
2. **Forward Flow:**
   User → Agent → Task Modules → Planner → Output
3. **Backward Flow:**
   Tool Feedback → Agent → Plan Adjustment
4. All communication is handled using structured data formats (JSON), ensuring consistency and clarity across modules.

Data Flow Diagram (DFD)



2.4 Component Design

1. Planner Agent

- Core intelligence of the system
- Uses LLM for reasoning and decision-making

2. Task Decomposition Module

- Converts high-level goals into smaller tasks
- Uses prompt engineering techniques

3. Tool Validator

- Simulates API/tool availability

- Ensures feasibility of tasks

4. Dependency Manager

- Identifies task relationships
- Ensures correct execution order

5. Execution Planner

- Generates final schedule
- Optimizes task sequence

2.5 Key Design Considerations

- **Scalability** → Modular design allows expansion
- **Flexibility** → Supports different marketing goals
- **Accuracy** → Uses structured reasoning loops
- **Efficiency** → Optimized task sequencing
- **Reliability** → Tool validation ensures feasibility
- **Maintainability** → Clean separation of components

2.6 API Catalogue

1. Input API

- Accepts user goal
- Method: POST
- Input: Goal (string)

2. Planning API

- Processes goal and generates plan
- Method: POST
- Output: Task list + schedule

3. Tool Validation API (Mock)

- Checks tool availability
- Input: Tool name
- Output: उपलब्ध / Not Available

4. Output API

- Returns final structured plan
- Format: JSON

3. References

- LangChain Documentation
- CrewAI Framework Guide
- OpenAI / LLM API Documentation
- Research papers on Agentic AI systems
- Prompt Engineering best practices